Documents in HTML, printed to PDF

papyrus is a documents engine. You write a document as HTML or React pages. On screen each page is a sheet of A4 or US Letter paper, and the same pages print to a PDF. A report or a resume stays one source for the web and for print.

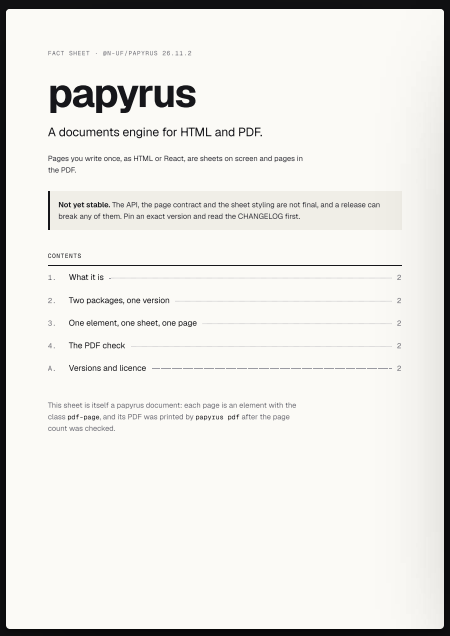
It checks every PDF before writing it: the number of pages in the HTML, the PDF's own page count and the count you expect must agree. If they don't, for example because a page overflowed onto an extra sheet, the export stops instead of writing the file.

papyrus is young. Versions are dates (YY.M.R), not semantic versions, so any release may break things. Pin an exact version and read the changelog before you upgrade.

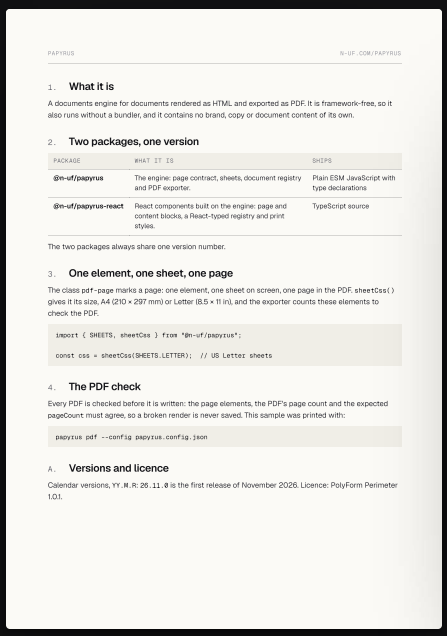
Two packages, one version

- **@n-uf/papyrus**, the engine. Plain ESM JavaScript with type declarations and no framework, so it runs in a plain HTML page, in a bundled app and in Node. It holds the page contract, the sheet sizes and print CSS, the document registry and the PDF exporter.
- **@n-uf/papyrus-react**, React components on top of the engine: page and content blocks, a document registry typed for React, and print styles. It ships TypeScript source.

Both carry no brand, copy or content of their own; your documents supply those. They are released together at one version, on npmjs with provenance and on GitHub Packages.



A4 · p. 1/2



A4 · p. 2/2



Letter · p. 1/1

papyrus fact sheet

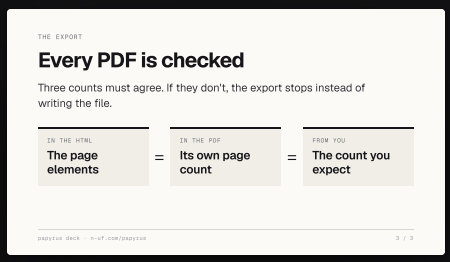
PDF · 2 pages · 169 KB



Slide 16:9 · 1/3



Slide 16:9 · 2/3



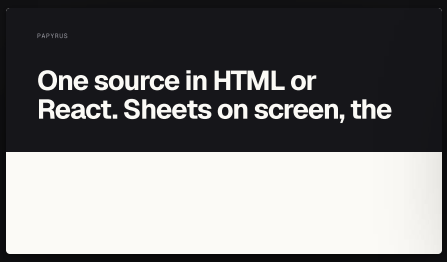
Slide 16:9 · 3/3

papyrus deck

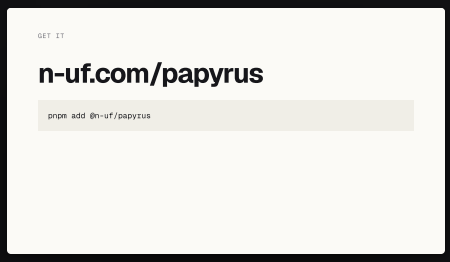
PDF · 3 slides · NaN KB



Slide 16:9 · 1/3



Slide 16:9 · 2/3



Slide 16:9 · 3/3

papyrus pitch

PDF · 3 slides · NaN KB

One element, one sheet, one page

Every element with the class `pdf-page` is one sheet on screen and one page in the PDF. `sheetCss()` sizes it to A4 or Letter and keeps one per printed page; on screen the sheets sit on a grey canvas.

```
import { SHEETS, sheetCss } from "@n-uf/papyrus";

const css = sheetCss(SHEETS.LETTER);                    // US Letter sheets
const alt = sheetCss(SHEETS.A4, { pageClass: "page" });  // your own page class
```

Pages without a bundler can link the ready-made `@n-uf/papyrus/sheet-a4.css` or `sheet-letter.css` instead.

Numbered tables of contents

A document is registered with its title, expected page count and table of contents. `numberToc()` numbers the entries, appendices included.

```
import { docRegistry, numberToc } from "@n-uf/papyrus";

docRegistry.register({ id: "report", meta: { title: "Report", pageCount: 4, toc },
component: render });
numberToc(toc); // Map: entry id -> "1.", "2.", "A." (appendix) or null (skipped)
```

Export to PDF

The `papyrus` command prints every document in a config file through headless Chromium (Playwright). The server that renders the documents must already be running (`--wait` gives it time to answer):

```
papyrus pdf --config papyrus.config.json
```

For plain HTML files no server is needed: point `baseUrl` at the folder.

```
{
  "baseUrl": "file:///Users/me/resume/src/",
  "docUrl": "{id}.html",
  "outDir": "../dist",
  "docs": [{ "id": "resume", "sheet": "letter", "pageSelector": ".page",
"pageCount": 3 }]
}
```

- **Sheet:** A4 by default, `"letter"` for the whole config or one document. It sets the viewport, the print CSS and the PDF paper.
- **Pages:** `pageSelector` names your page elements when they are not `.pdf-page`; `pageCount` is the number you expect.
- **Fonts:** web fonts are awaited before printing (up to 10 seconds), so pagination doesn't depend on load timing.
- **Stale PDFs:** list a document's source folders and `papyrus pdf --check` reports committed PDFs whose sources changed since they were printed.
- **Browser:** set `PAPYRUS_CHROMIUM_PATH` to print with an installed Chrome instead of Playwright's download.

The same export runs from code with `renderPdf(browser, url, options)` from `@n-uf/papyrus/pdf`. `createPdfHandler` from `@n-uf/papyrus/server` renders a document to a downloadable PDF on request, in a serverless function or any server with a Web `Request`. In development, the Vite plugin `papyrusPdfPlugin` from `@n-uf/papyrus/vite` serves `/docs/<id>.pdf`.

React components

`@n-uf/papyrus-react` gives you `PDFPage` and blocks for text, tables, lists, cards and indicators. `registerDoc` (from `/dossier`) registers a React document, `useDocParam` (from `/params`) keeps a document parameter in the URL, and `printStyles` (from `/styles`) holds the sheet CSS and print fixes. Branding is yours: `Logo` and `DocumentFooter` render only what you pass.

```
<Logo name={<MyWordmark />} variantTextLines={{ home: ["MADE BY"] }} />
<DocumentFooter title="Report" copyright="© 2030 Acme" />
<TOCItem number="1" title="Intro" page="3" section="market" sectionColors={{ market:
"rgb(14, 116, 144)" }} />
```

With Tailwind v4, import the package's stylesheet: `@import "@n-uf/papyrus-react/tailwind.css"`.

Install

```
pnpm add @n-uf/papyrus            # engine only, no bundler needed
pnpm add @n-uf/papyrus-react      # React components
```

In a Next.js app, add `@n-uf/papyrus-react` to `transpilePackages`. `playwright` and `vite` are optional peer dependencies, needed only for the PDF export and the Vite plugin.

Source on GitHub: n-uf/papyrus. On npm: @n-uf/papyrus and @n-uf/papyrus-react. Changelogs: engine, React components. Licence: PolyForm Perimeter 1.0.1.